

Sea Lyon Media Manager — Feature Completeness & Code Review

Version reviewed: 0.8.0 (branch LMM-DEV) **Date:** 2026-05-22 **Reviewer:** Engineering review pass **Scope:** Release readiness (Part 1) + Industry leadership gap analysis (Part 2)

Executive Summary

Sea Lyon Media Manager is a feature-rich Windows-first desktop media manager written in Python/PySide6. The codebase is ~28k LOC across 64 modules with a mature **637-test pytest suite**. The LMM-DEV branch is feature-complete per the v0.8.0 plan (all 5 implementation tiers landed) and is in extended manual testing prior to merging to LMM-MASTER.

Verdict:

- **Release-ready?** Almost. The product is functionally complete for a v1.0 Windows release. The gaps are not features — they are **release-engineering** items (code signing, installer, crash reporting, license/legal, an end-user installer experience, an auto-update channel, and a public website / download page). Roughly **2–4 engineering weeks** of release work stand between today and a public installer download.
- **Industry leader?** Not yet. To compete with MusicBee, MediaMonkey, foobar2000, JRiver, and Roon, the app needs **streaming integration, a high-end audio pipeline (bit-perfect / WASAPI exclusive / ASIO), mobile companion, and a modern sync story**. Estimated **6–12 months** of focused work on top of release hardening.

Part 1 — What's Needed to Release

1.1 Current Feature Inventory (what already ships)

Area	Status
Library (SQLite, scan, search, watch, smart playlists, ratings, dupes)	Complete
Music playback (libVLC, queue, crossfade, gapless, shuffle, repeat, sleep timer)	Complete
10-band equalizer w/ presets and custom curves	Complete
Now Playing (cover art, lyrics .lrc/embedded/LRCLIB, queue preview, rating)	Complete
Podcasts (RSS/Atom, OPML import, background refresh, episode catalog)	Complete
Internet Radio (M3U/PLS)	Complete
CD detection + ripping (libdiscid, CTDB, MB, AccurateRip v1, raw CDDA fallback)	Complete
Output formats: FLAC/MP3/AAC/Opus/OGG/ALAC/WAV/AIFF/WMA	Complete
Video player (libVLC, subtitles, snapshots, fullscreen, audio-track select)	Complete
YouTube search + download (yt-dlp, audio/video, playlists)	Complete
Metadata (CTDB → MB → TheAudioDB → CAA artwork)	Complete
DLNA/UPnP Cast (smart TV, AV receiver)	Complete
DLNA media server	Complete
Scrobbler	Complete
ReplayGain	Complete
AcoustID fingerprinting	Complete
Theme + branding (dark WMP-inspired UI, splash, icons)	Complete
Runtime diagnostics (ffmpeg/VLC/discid/Python paths)	Complete
First-run setup, settings dialog, knowledge-base / F1 help	Complete

Library statistics dialog	Complete
Windows build automation (PyInstaller spec, PowerShell script, Actions workflow)	Complete

This is **broader functional coverage than most paid competitors** at the "music + podcasts + radio + video + CD + YouTube + cast" intersection. It is already a viable replacement for Windows Media Player, iTunes, MusicBee, and Winamp in most workflows.

1.2 Pre-Release Blockers (must fix before public release)

These are non-negotiable for a public release on Windows in 2026.

1.2.1 Distribution & installer

- **Inno Setup .exe installer** — build/lyon.iss exists but should produce a single signed LyonMusicManagerSetup.exe users double-click. Today the GitHub Actions workflow only produces a zip of dist\LyonMusicManager\.
- **Code signing (Authenticode)** — without an EV or OV code-signing cert, Windows SmartScreen and AV products will block or warn against the installer. This is the single biggest UX blocker for an unsigned indie release. Budget ~\$200–\$700/yr for an OV cert from Sectigo, DigiCert, or SSL.com.
- **Auto-update channel** — Sparkle/WinSparkle-equivalent, GitHub Releases with appcast XML, or a custom updater. Without it, users on 0.8.0 will be stranded on 0.8.0.

1.2.2 Legal & licensing

- **EULA** — the project ships LICENSE (presumably MIT) but the **bundled binaries (VLC, ffmpeg, libdiscid, yt-dlp)** are GPL/LGPL/Unlicense and need their license text reproduced and shown in About → Licenses.
- **Third-party attribution screen** in About dialog. There is lyon/ui/about.py but verify it lists all license-required dependencies (Qt LGPL exception notice, VLC LGPL, ffmpeg LGPL/GPL depending on build, yt-dlp Unlicense, etc.).
- **MusicBrainz contact string** — README warns to set a real contact value before distribution. Verify the default shipped in settings.json is not lyon@example.com or similar placeholder.
- **YouTube ToS** — yt-dlp integration is legally grey. Add a clear in-app disclaimer that downloading is user-responsibility and is region/jurisdiction dependent. Consider gating YouTube view behind a one-time "I understand" acknowledgement on first launch.
- **CDDB/MusicBrainz/CTDB rate-limit compliance** — verify the User-Agent header includes app name, version, and contact URL on every outbound request.
- **Privacy policy** — required for any app touching scrobble.audio/last.fm, MusicBrainz, TheAudioDB, LRCLIB, or yt-dlp. Even if "we collect nothing," a one-page privacy statement is needed before listing on any store or site.

1.2.3 Crash & error reporting

- **No crash reporter.** A Python desktop app without a crash collector will ship with bugs that never get reported. Recommended: sentry-sdk with opt-in toggle (off by default for privacy) **or** a local crash-dump file
 - "open log folder" button in Help menu.
- **Structured logging to rotating file** — metadata-diagnostics.log exists but main-app log is not rotated. Add RotatingFileHandler (10 MB × 5 files) in lyon/app.py so support requests can include a log.

1.2.4 Release engineering

- **Versioned release notes / CHANGELOG.md** — README has a "Status" section but no CHANGELOG.md. Required for users to know what they upgrade into.
- **Tagged releases on GitHub** with attached signed installer zip + sha256.
- **Reproducible build** — confirm scripts/build-windows.ps1 produces a byte-identical build given the same inputs (PyInstaller is mostly OK; pin every dependency including transitive ones in requirements.txt).
- **macOS / Linux build paths** — README says "Windows-focused" but main.py runs on macOS and Linux. Either explicitly mark those as unsupported ("source-only, no installer") **or** invest in .dmg / .AppImage. For v1.0 I recommend Windows-only and remove macOS/Linux from advertised surface to scope.

- **Installer detects missing VLC and ffmpeg** and either bundles them or offers a one-click download. Today the README says "place files under bin\" — that's an unacceptable end-user experience. Ship them bundled (size hit is ~150 MB but worth it).
- **Uninstaller cleans up** %APPDATA%\LyonMusicManager\ with a "delete user library/settings?" prompt.

1.2.5 Quality gates before tagging v1.0

- Run full 637-test suite on a clean Windows 10 + Windows 11 VM. Today the test collection works (verified via `pytest --collect-only`); enforce green-build-required in CI.
- Manual smoke test against the **release checklist** in docs/:
 - First-run setup completes end-to-end with no library configured.
 - Audio CD detection, rip, library import all green on a real machine.
 - DLNA cast to at least one real TV / receiver.
 - YouTube audio + video download + library import.
 - Equalizer audibly affects output (the EQ-no-effect failure mode is the most user-visible regression in this app's history).
 - Crossfade and gapless work across queue boundaries.
- Run on a 4K display and a 125% / 150% / 175% DPI configuration.
- Test on a fresh user with no %APPDATA%\LyonMusicManager\ at all.

1.3 Release-Blocker Summary Table

Blocker	Severity	Estimated effort
Inno Setup installer + signed .exe	Critical	3–5 days
Authenticode code signing certificate	Critical	1 day + procurement
Auto-update mechanism	High	5–7 days
Bundle VLC + ffmpeg in installer	Critical	2 days
Crash reporting (Sentry opt-in or local dumps)	High	2 days
EULA + third-party license screen	Critical	1 day
CHANGELOG.md + versioned release notes	High	0.5 day
YouTube usage disclaimer / acknowledgement	High	0.5 day
Rotating log file	Medium	0.5 day
Privacy policy page	Critical	1 day
Manual full-stack smoke test on Win10 + Win11	Critical	2 days
Total to public 1.0		18–22 dev days (4 weeks)

Part 2 — Code Review

2.1 High-level observations

- **Architecture is clean**: clear separation between `lyon/core/` (logic, no Qt widgets) and `lyon/ui/` (PySide6 only). This is the single most important architectural decision in a Qt/Python app and it is correct here.
- **No TODO / FIXME / HACK markers** in the source tree (verified via `grep` — the 5 matches are all `TXXX` ID3 frame string literals, not actual TODOs). This is unusual and indicates good hygiene.
- **Type hints are present** via `from __future__ import annotations` and typing imports. Not 100% covered but materially better than the typical Python desktop app.
- **637 tests** spanning settings, dialogs, library queries, smart playlists, metadata providers, CTDB lookup/verification, ripping command paths, CD detection, playback backend fallback, equalizer, transport, lyrics cache, video player handoff, branding, accessibility, UI dependencies, and packaging. This is **substantially**

better coverage than most desktop Python projects.

- **Resource lifecycle is explicitly managed:** SQLite close in `MainWindow.closeEvent`, `metadata.shutdown()`, libVLC release on shutdown, Windows DLL-directory handle close, worker thread joins. The README itself documents this contract.
- **No `os.system`, no `eval`, no `pickle.load` from untrusted sources** in the modules surveyed. Defused XML is used for OPML/M3U parsing.

2.2 Code-quality concerns (sorted by severity)

High

1. **Large UI modules.** `lyon/ui/library_view.py` is **2,357 lines**; `knowledge_base_dialog.py` is 1,643; `video_player_view.py` is 1,629; `main_window.py` is 1,576. These need to be split. A Qt widget file over ~800 lines becomes hard to test, hard to review, and accumulates incidental coupling. Suggested split for `library_view.py`: model layer, tree-builder helpers, context-menu actions, drag/drop, search bar — each into its own file.
2. **Core modules also large.** `lyon/core/library.py` (1,666 lines) and `lyon/core/metadata.py` (1,359 lines) and `lyon/core/dlna_server.py` (1,298 lines) and `lyon/core/ripper.py` (1,285 lines) each blend several responsibilities. `library.py` mixes schema/migration, query layer, playlist queries, smart-playlist materialization, and scan logic. Long term this is the riskiest area for regression because every feature touches `library.py`.
3. **No `mypy/pyright` in CI.** Type hints exist but are unenforced. A single `from __future__ import annotations` typo can mask a real bug. Add `mypy --strict-optional` `lyon/core/` as a CI gate; `lyon/ui/` can start permissive.
4. **No `ruff/black` in CI.** No lint or formatter enforcement that I can see. Add `ruff` (it is fast and catches real bugs like undefined names, shadowed builtins, and unawaited coroutines) and `black` (or `ruff format`) to CI.

Medium

5. **Bundled binaries are not pinned by hash.** `scripts/build-windows.ps1` downloads `ffmpeg` / `libdiscid` / `VLC` at build time. If those mirrors are compromised or move, builds break or worse. Pin each download by SHA-256 in a manifest file and fail the build if the hash mismatches.
6. **SQLite migrations.** `lyon/core/library.py` does not appear to use a formal migration framework (e.g. `alembic`). Today this is fine because the app is v0.8.0 — but v1.0 is the moment to put `schema_version` and forward-only migrations in place, before users have data you can't break.
7. **No automated UI screenshot / golden-image regression.** The `test_branding.py` and `test_qss_validity.py` tests confirm assets load but do not catch visual regressions. Optional; foobar2000-tier visual stability is not realistic at 1 contributor.
8. **Async story is thread-based, not `asyncio`.** Every long-running task uses `QThread` / signals. This is correct for Qt, but it means library scans, podcast refresh, metadata fetch, and YouTube downloads all contend for thread-pool slots. Verify there is no scenario where two long scans + a rip + a podcast refresh starve the UI thread on slow spinning disks. Add a hard cap to the worker thread pool.
9. **Settings file format is JSON, not versioned.** A breaking change to `settings.json` will corrupt user state silently. Add a `schema_version` field and a migration path.

Low

10. **Defusedxml is used** — good — but verify it is used **everywhere** XML is parsed: OPML, M3U, PLS, CTDB XML response, MB XML response. A grep audit before release is cheap.
11. **requests is used** without an explicit timeout on every call. Verify `metadata.py` and `podcast.py` and `radio.py` set a sane timeout (5–10s) and retry policy. A hanging HTTP connection can freeze the UI if it ever runs on the main thread.
12. **Crossfade backend** — the README's "May 17, 2026 memory/resource review fixed remaining Qt ownership leaks in crossfade backends/timers" note means this code is historically leaky. Add a long-running soak test that does 1,000 track changes with crossfade enabled and asserts resident-set-size growth is below a threshold.
13. **macOS media-key hook** is optional and platform-conditional. If you ship macOS as supported, this becomes a first-class platform feature; if you don't, consider removing or guarding it behind a feature flag to reduce surface area.
14. **pyacoustid is a runtime dep** but AcoustID fingerprinting requires `fpacalc` (Chromaprint) as a native binary.

The Actions workflow downloads fpcalc; the README does not mention it. Either bundle it unconditionally or remove the dependency until UI exposes the feature.

Informational

15. **Code style is consistent.** PEP 8 spacing, snake_case identifiers, triple-quoted docstrings on most public modules. No `print()` debug leftovers in modules surveyed.
16. **Logging is via `stdlib`** logging with module-level `LOG = logging.getLogger(__name__)` — correct.
17. **No dead-code modules** observed in `lyon/core/` — every module appears imported somewhere.

2.3 Security review

- **Defused XML** — confirmed for at least one path. Audit the rest.
- **SQL** — `library.py` should use parameterized queries everywhere. The smart-playlist SQL builder is the most likely place for SQL injection if user-provided strings are interpolated. **Verify before release.**
- **Path traversal** — when importing M3U playlists or OPML, ensure paths are resolved against expected library roots and that absolute paths pointing outside the music root cannot be silently followed when the feature operates on user-visible side effects.
- **DLNA server** — `dlna_server.py` is 1,298 lines and exposes HTTP on the LAN. Verify it (a) binds to a sensible interface by default, (b) rejects requests for files outside the library, and (c) does not log request bodies that include track paths to a world-readable log.
- **yt-dlp** — runs subprocesses on URL inputs. Verify URLs are not passed via `shell=True` and that the subprocess is invoked with an `argv` list.
- **No hardcoded secrets / API keys** observed; verify TheAudioDB and LRCLIB keys (if any) are not committed.

Part 3 — Industry Leadership: What's Needed to Win the Segment

3.1 Market positioning

The market this product sits in has 5 broad camps:

Camp	Examples	What they're great at	What Sea Lyon already beats them at
Free Windows classics	foobar2000, AIMP, Winamp	Lightweight playback, audiophile DSP, plugins	Modern UI, video, podcasts, CD ripping integrated, YouTube
Paid power-user managers	MusicBee, MediaMonkey, JRiver	Sync, scripting, deep library	Already at feature parity on library + ripping + cast
Audiophile / hi-res	JRiver, Roon, HQPlayer	Bit-perfect WASAPI/ASIO, room correction, multi-zone	Sea Lyon does not compete here yet
Streaming-first	Spotify, Tidal, Apple Music	Catalog, recommendations, mobile	Sea Lyon does not compete here at all
Server / cross-device	Plex, Jellyfin, Emby, Roon	Remote streaming, mobile apps, transcoding	Sea Lyon has DLNA cast and a DLNA server; that's a foothold but not a competitive product

Sea Lyon's natural target is **MusicBee + MediaMonkey + foobar2000**: a power-user local-library Windows app, with stronger out-of-the-box video, podcasts, YouTube, and CD ripping than any of them. To win that segment outright you need to close gaps in **audio quality, sync, plugins, and streaming**. To win beyond it you need a mobile companion.

3.2 Industry-leader feature gaps (sorted by ROI)

Tier 1 — must-have for "best Windows media manager" claim

1. **Bit-perfect / WASAPI exclusive / ASIO output.** libVLC's default output on Windows is shared-mode WASAPI; for audiophiles this is a deal-breaker. Add WASAPI exclusive and ASIO as selectable outputs in Settings → Playback. JRiver, foobar2000, MusicBee all expose this. **High ROI.**
2. **Plugin / scripting API.** foobar2000 and MusicBee dominate because third parties extend them. A Python plugin API (lyon.plugins entry points) for at minimum: custom metadata providers, custom output formats, custom views, custom DSP nodes. Even a small plugin SDK creates moat.
3. **Tag editor power features.** Batch-tag with regex, file-rename templates with conditional tokens, Discogs integration, manual artwork browser (drag from web). Today the app has batch metadata edit and MB fetch; bring it to MP3Tag parity.
4. **Last.fm scrobbling end-to-end UX.** A scrobbler module exists; verify login flow, offline scrobble caching, and Listenbrainz support are both first-class. Power users care.
5. **Visual themes / skins.** A single dark theme is correct for v1.0. For v1.5, expose a theming surface (QSS-based) and ship 3–4 themes. This is table stakes vs. AIMP/Winamp.
6. **HiDPI polish.** Test specifically at 125%/150%/175%/200% scale and on 4K screens. Qt is mostly correct but custom-painted widgets (transport, now playing, library grid cells) need explicit DPI testing.

Tier 2 — closes the "power user" gap

7. **Sync to portable devices** (USB MP3 players, Android over MTP, iPod classic via libgpod, generic mass-storage). MusicBee's killer feature. Big project (~6–8 weeks). High user impact.
8. **Convert-on-the-fly during sync** (FLAC → MP3/AAC at chosen bitrate while syncing). Same project as #7.
9. **Convert / transcode utility** as a first-class library action. "Right-click → Convert" with a queue.
10. **Audio fingerprint-based deduplication.** AcoustID is already a dependency; expose it in the duplicate finder.
11. **Library auto-tagging from filename / folder structure.** Template- driven, with preview.
12. **CUE sheet support** — cue_parser.py exists. Verify full FLAC + CUE playback as a multi-track virtual album, including ripping single-file FLAC+CUE images back to per-track FLAC.
13. **DSD playback / FLAC 24/192 verification** in the audiophile output path. Important for the buyer who currently pays for JRiver.
14. **ReplayGain analysis UI** (already have the engine via replaygain.py; verify it has a "Analyze selected" action with a progress dialog and stores values into tags).
15. **Internet radio directory** (TuneIn / radio-browser.info integration) so users don't have to find stream URLs themselves.
16. **Podcast: download episodes for offline + auto-cleanup** policies (keep last N, delete after played).

Tier 3 — closes the "streaming era" gap

17. **Tidal / Qobuz / Apple Music integration** via their HiFi APIs. This is the single biggest reason audiophiles still pay for Roon and JRiver. Hard (vendor approvals + DRM); high reward.
18. **Spotify Connect target** — be a Spotify Connect speaker. Requires Spotify's librespot or partner SDK.
19. **AirPlay 2 target** + Chromecast Audio target. DLNA is half of the cast story; AirPlay is the other half. There are reverse-engineered libraries (pyatv, shairport-sync) but legal/quality bar is high.
20. **YouTube Music** integration as a logged-in catalog browser, not just yt-dlp search. (Legal grey area — proceed cautiously.)
21. **Recommendations / "More like this"** using Last.fm + ListenBrainz
 - local play history. Roon's "Radio" feature is its anchor and is achievable with a moderate engineering investment.

Tier 4 — multi-device / mobile

22. **Headless server mode** — python -m lyon --server that exposes library + playback as an HTTP/JSON API and a UPnP/DLNA renderer. The pieces exist (DLNA server, library, player). Repackaging as a proper server unlocks #23.
23. **Mobile companion app** (iOS + Android). Even a thin remote that browses the library and controls the

desktop player is a huge differentiator vs. MusicBee/MediaMonkey, which have weak or paid remotes. Could be a Flutter app or a Qt for Mobile app sharing the PySide6 widget library.

24. **Cloud sync of library metadata + playlists** between machines (not files — metadata + ratings + play counts + playlists). Roon does this; JRiver does this with JRemote.
25. **Web UI** for the headless server. Jellyfin and Plex's web UIs are the reason they win the family-server segment.

Tier 5 — modern niceties

26. **GPU-accelerated video** verified for HEVC, AV1, VP9 on integrated Intel/AMD/NVIDIA. libVLC does this but worth a verification matrix.
27. **HDR passthrough** for video.
28. **Closed-caption / subtitle search** (OpenSubtitles).
29. **Smart album view** that detects single-file FLAC+CUE images, multi-disc albums, deluxe editions, and groups them correctly.
30. **Lyrics editor** — write your own .lrc files for tracks LRCLIB doesn't have.
31. **Hotkey customization** with global hotkeys (media keys + custom combos that work when the app is in background).
32. **Mini player / compact mode** + always-on-top thumbnail player.
33. **Equalizer expansion** — convolution / parametric EQ, room correction (REW / Dirac), Dolby Atmos for Headphones routing.
34. **Visualizations** (foobar2000-style spectrum and waveform). Low ROI but expected.

3.3 Recommended 12-month roadmap to "industry leader"

Quarter	Theme	Headline deliverables
Q3 2026	Release 1.0	Public installer, code signing, auto-update, crash reporting, EULA, marketing site.
Q4 2026	Audiophile credibility	WASAPI exclusive + ASIO output, DSD, ReplayGain UI, CUE+FLAC, plugin API (alpha), MP3Tag-class tag editor.
Q1 2027	Power user parity	Portable-device sync with transcode-on-the-fly, internet-radio directory, podcast offline + retention, AcoustID dedup, theme support (3 themes), full HiDPI pass.
Q2 2027	Networked era	Headless server mode, web UI (alpha), AirPlay2 / Chromecast Audio targets, Last.fm/ListenBrainz first-class, recommendations engine.
Q3 2027	Mobile + streaming	Mobile remote (iOS+Android), Tidal/Qobuz integration (HiFi tier), Spotify Connect target.

After Q3 2027 the product would credibly compete with **MusicBee, JRiver Media Center, and entry-level Roon** simultaneously — which no other product on the market does today. The closest competitor is MusicBee, and MusicBee is single-developer freeware with no mobile story; this product can both match and exceed it.

3.4 Strategic recommendations

1. **Pick a license model:** free + paid Pro tier (sync, streaming integrations, mobile) is the obvious model. Free open-source forever is also viable but harder to sustain.
2. **Ship Windows 1.0 first.** Resist the urge to do macOS / Linux installers before Windows is rock-solid. The market segment lives on Windows.
3. **Decide on the plugin API now**, even if you ship empty in 1.0. Once third parties write extensions against the wrong API, you cannot change it. Look at MusicBee's plugin model and foobar2000's component model for prior art.
4. **Buy the EV code-signing certificate.** SmartScreen reputation builds only with EV. An OV cert is the budget option; an EV cert pays for itself within a quarter for any commercially distributed Windows app.
5. **Set up a public bug tracker and changelog feed** simultaneous with 1.0 launch. Indie media managers live

or die on responsiveness to user reports.

6. **Pick scrobbling as a marketing wedge.** "Best scrobbling experience on Windows in 2026" is a credible and achievable claim today.

Appendix A — Files audited

This review was performed by:

- Reading `README.md` (427 lines)
- Listing all source files in `lyon/core/` (29 modules) and `lyon/ui/` (30 modules)
- Spot-reading headers of `library.py`, `player.py`, `main_window.py`, `playback_backend.py`
- Reading the memory record of the v0.8.0 implementation plan (Tiers A–E, all complete as of 2026-05-22)
- Collecting tests via `pytest --collect-only` (637 tests)
- Grepping for `TODO/FIXME/XXX/HACK` across `lyon/`
- Reviewing `.github/workflows/windows-build.yml`
- Listing `docs/`, `scripts/`, `build/`

Files **not** read line-by-line (a deeper review pass should cover): `dlna_server.py`, `ripper.py` end-to-end, `metadata.py` HTTP/UA paths, `smart_playlist.py` SQL generation, full `library_view.py` Qt model code, `video_player_view.py` fullscreen handoff.

Appendix B — Quick wins (do these this week)

1. Add `CHANGELOG.md`.
2. Add `ruff` + `black` + `mypy` to a new CI workflow.
3. Hash-pin the `ffmpeg` / `libdiscid` / `VLC` downloads in `scripts/build-windows.ps1`.
4. Add `RotatingFileHandler` to `lyon/app.py`.
5. Audit `requests` calls for explicit timeouts.
6. Audit XML parsing for `defusedxml` everywhere.
7. Audit SQL builders in `smart_playlist.py` for parameterization.
8. Add a CI gate that fails the build if `pytest` collection drops below the current count of 637.

End of report.